

ARGUS

Autonomous AI QA Agent

Crawls . Plans . Generates . Runs . Triages . Files

By Everton S. Andrade

Document dated: August 23, 2026

1. Overview

Argus is an autonomous AI QA agent that crawls a web application, uses Claude to plan a prioritized test suite, generates 100% deterministic Playwright tests using a template-first approach, executes them, and then uses Claude again to triage failures -- distinguishing real bugs from flaky tests, stale selectors, and environment issues.

Key distinction: Argus is the GENERATION + TRIAGE half of the autonomous QA loop. Its sibling project Cerberus CI is the REGRESSION GATE that locks in existing test suites and prevents regressions on main.

How It Works

1. Ingest

Crawl the app, discover interactive elements, build feature inventory

2. Plan

Claude proposes a prioritized test suite from the feature inventory

3. Codegen

Templates first (30+ deterministic rules), LLM only as fallback

4. Execute

Runs the generated Playwright suite, captures failures with DOM snapshots

5. Triage

Claude classifies each failure: real_bug / flaky / selector_drift / environment

6. File

Creates GitHub Issues with severity labels + inline PR annotations

7. Report

Severity-based CI gate + dashboard data

AI Call Budget: 2 + N per run (1 planning call + 1 triage call per unique failure). Everything else is deterministic and free.

2. Key Features

2.1 Templates First, LLM Second

src/codegen/templates.ts contains 30+ deterministic template rules covering:

- Navigation (navigate-to-route, reload)
- Interaction (click-text, click-selector, check-nth, check-selector, fill, select)
- Assertions (assert-text, assert-checked, assert-not-checked, assert-should-see, assert-should-not-see)
- Security (require-https, reject-empty-input, validate-email-format)
- Accessibility (accessible-name, keyboard-navigable, no-autoplay-media, color-contrast)

The LLM is only called if no template matches -- and in 100% of demo runs, templating covered everything.

2.2 Verdict Cache

Error signatures are normalized (strip line:col, UUIDs, timestamps, URLs) and hashed (SHA-256, 16-char hex):

Example normalization:

"AssertionError at src/page.ts:42:17 in https://api.com/u/550e8400"

-> "AssertionError at in api.com u"

-> SHA-256 -> 16-char hex key

30-day TTL per entry, cached in data/runs/<runId>/trriage-cache.json. Repeated failures skip the AI call entirely -- up to 60% cost reduction on apps with flaky tests.

2.3 Severity-Based Gate

The CI gate is NOT pass/fail -- flakiness never blocks merges:

CI FAILS only if: new real bugs at or above severityFailThreshold

CI PASSES if: all failures are flaky / selector_drift / environment

OR all real bugs are duplicates of existing issues

2.4 Provider Abstraction

The AIProvider interface abstracts LLM calls, mirroring Cerberus's pluggable adapter design:

- ClaudeProvider (Anthropic API)
- OpenAICompatibleProvider (OpenAI, DeepSeek, Gemini, Ollama)
- MockProvider (for testing, no API key needed)

Config-driven via ARGUS_AI_PROVIDER env var. Supports OpenAI Compatible endpoints with ARGUS_OPENAI_BASE_URL.

2.5 Zero API Keys for Testing

- npm test -- 313 tests, all fixture-based with MockAiClient
- npm run run:mock -- full pipeline demo with mocked AI (no API key needed)

3. Architecture

3.1 Deduplication Signature

```
type BugSignature = {
  title: string;    // normalized test title
  featureId: string; // what was being tested
  errorClass: string; // e.g., "AssertionError" (NOT lowercased!)
  verdict: Verdict; // real_bug / flaky / selector_drift / environment
}
```

Weighted scoring: title (base) + featureId (+0.15) + matching verdict (+0.10)

Penalty: -0.55 for mismatched errorClass or verdict. Threshold: 0.55

3.2 Project Structure

```
argus/
+-- src/
|  +-- cli/           # CLI entry, pipeline orchestration, CI reporting
|  +-- shared/        # Types, config, AI provider abstraction
|  +-- ingestion/     # Crawl web app, build feature inventory
|  +-- planner/       # AI stage: generate test plan from features
|  +-- codegen/       # Deterministic codegen (0 AI calls)
|  +-- execution/     # Run Playwright, capture failures
|  +-- triage/        # AI classification + verdict cache
|  +-- bug-filer/     # GitHub Issues filing + dedupe
+-- dashboard/       # Next.js 15 + Tailwind dashboard
+-- demo-app/        # Demo app with known bugs
+-- test/            # 313 tests (all fixture-based)
+-- docs/            # Demo assets, interview talking points
```

3.3 CI Gate Logic

gate = "fail" IF any NEW real_bug at or above severityFailThreshold

gate = "pass" IF all failures are flaky / selector_drift / environment

OR all real bugs are duplicates of existing issues

4. Running Argus

4.1 Quick Start (No API Key)

```
git clone https://github.com/EvertonSt/argus
cd argus
npm install
npm test          # 313 tests, no API key needed
npm run run:mock   # full pipeline demo with mocked AI
```

4.2 Run Against Your App

```
npx argus run --target http://localhost:3000
```

4.3 Serve the Dashboard

- npm run dashboard -> local dev server
- npm run dashboard:build -> static export for Vercel

4.4 Configuration

All env-driven:

ARGUS_AI_PROVIDER	claude	claude, openai, ollama, mock
ARGUS_AI_MODEL	claude-sonnet-4	Planning + triage model
ARGUS_TARGET	localhost:4317	App to test
ARGUS_GITHUB_TOKEN	--	GitHub token (absent -> dry-run)
ARGUS_GITHUB_REPO	--	owner/repo for Issues
ARGUS_GATE_SEVERITY	high	Min severity for CI gate failure
ARGUS_MAX_AI_CALLS	100	Total AI call cap
ARGUS_BROWSER	chromium	Playwright browser

5. Portfolio Context

5.1 Project Lineage

Project	Role	Focus	Tests
Argus	Solo founder	Generation + Triage	313
Cerberus CI	Solo founder	Regression gate	237

Argus creates the test suite -- it's the autonomous agent that discovers, plans, generates, and triages. Cerberus CI gates existing suites -- it's the deterministic CI gate that prevents regressions. Together they form a complete autonomous QA loop: generate -> triage -> gate -> repeat.

5.2 Tech Stack

- Primary: TypeScript, Node.js, Playwright
- AI: Claude (Anthropic API), OpenAI-compatible providers
- Dashboard: Next.js 15, Tailwind CSS, Chart.js
- Deployment: Vercel
- Testing: Vitest (313 fixture-based tests)
- License: MIT
- Node: 20+, TypeScript 5.6+

For full details, source code, and live demos, visit:
<https://github.com/EvertonSt/argus>